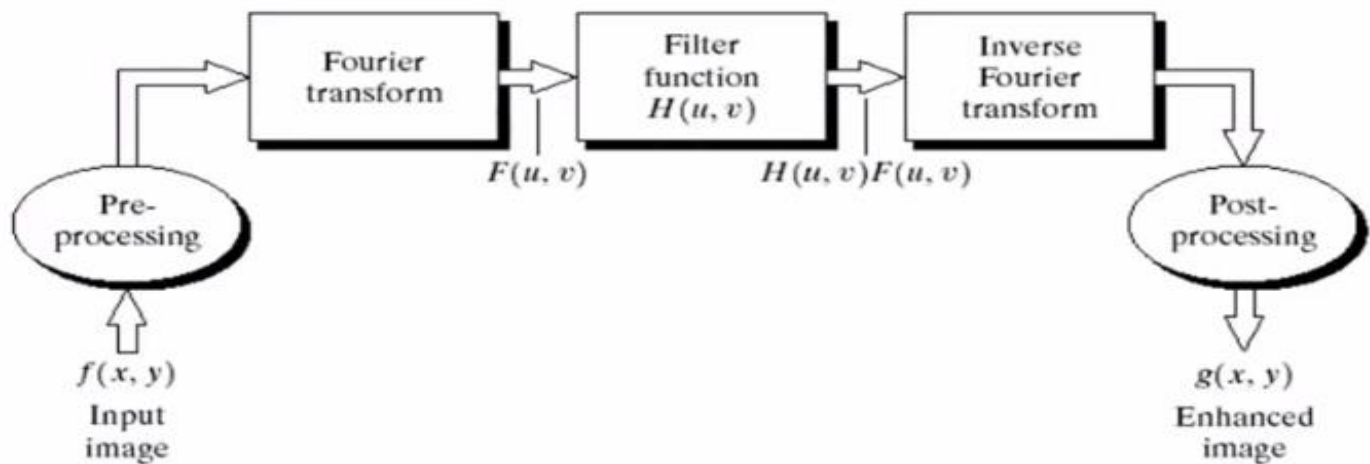


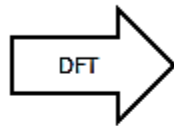
Frequency Domain Enhancement



1. Fourier transform

$$F(U, V) = \mathcal{F}(f(x, y)) = \frac{1}{NM} \sum_{Y=0}^{M-1} \sum_{X=0}^{N-1} f(x, y) e^{-2\pi i (\frac{UX}{N} + \frac{VY}{M})}$$

0	5	10
5	0	20
4	8	2



54	-13.5+16.4j	-13.5+16.4j
-4.5-9.5j	15+1.7j	-24+5.1j
-4.5+9.5j	-24-5.1j	15-1.7j

- Magnitude:

$$\|F(U, V)\| = \sqrt{R^2(U, V) + I^2(U, V)}$$

- Phase

$$\phi(F(U, V)) = \tan^{-1} \frac{I(U, V)}{R(U, V)}$$

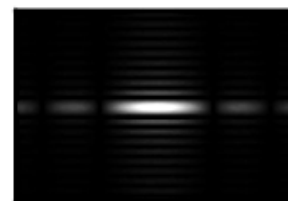
All the information we need about the image geometric structure is presented by magnitude.



original image



DFT (zero frequencies represented in the corners)



DFT (zero frequencies represented in the center)

2. Filter function

$$D(u, v) = \sqrt{\left(u - \frac{m}{2}\right)^2 + \left(v - \frac{n}{2}\right)^2}$$

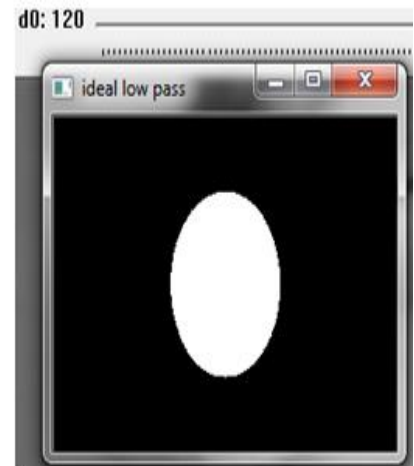
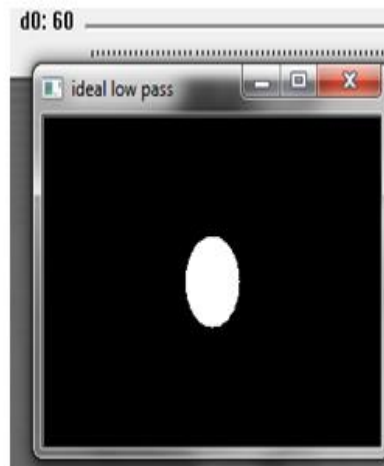
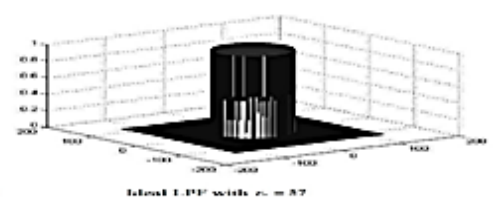
Low pass:

$$H(u, v) = \begin{cases} 1 & \text{if } D(u, v) \leq d_0 \\ 0 & \text{if } D(u, v) > d_0 \end{cases}$$

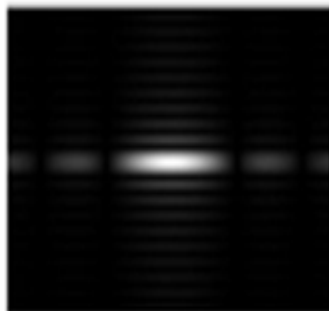
High pass:

$$H(u, v) = \begin{cases} 0 & \text{if } D(u, v) \leq d_0 \\ 1 & \text{if } D(u, v) > d_0 \end{cases}$$

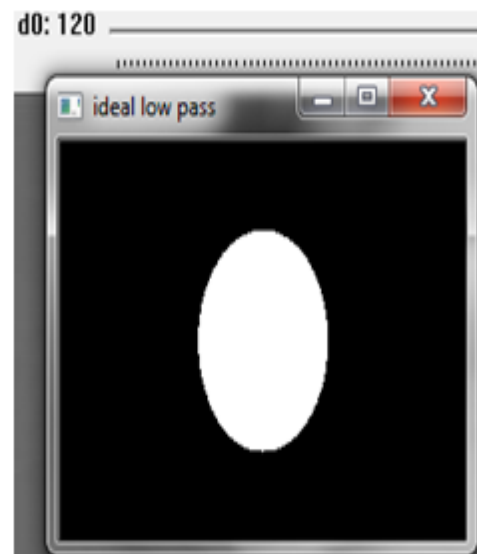
LOW PASS FILTERING



- Apply filter by multiplying DFT image $F(u, v)$ with a filter function $H(u, v)$

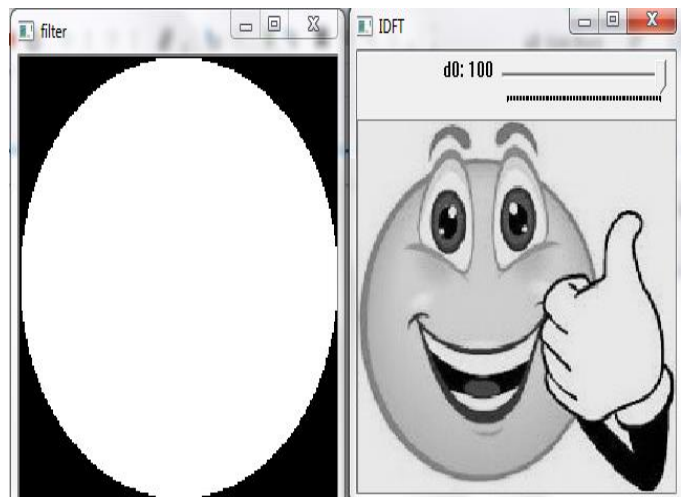
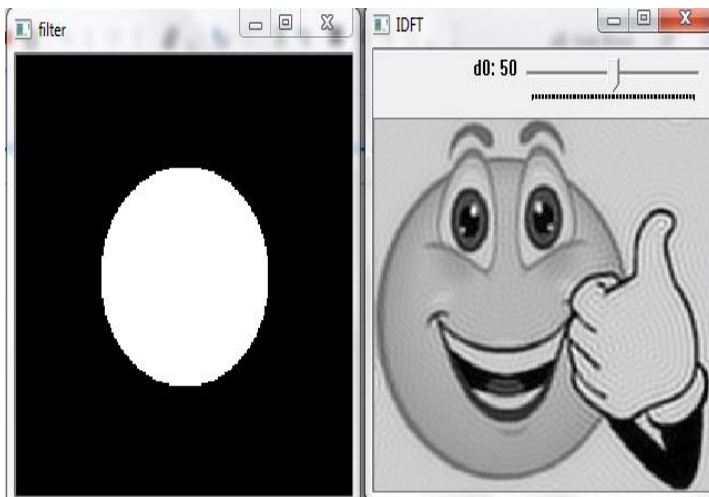
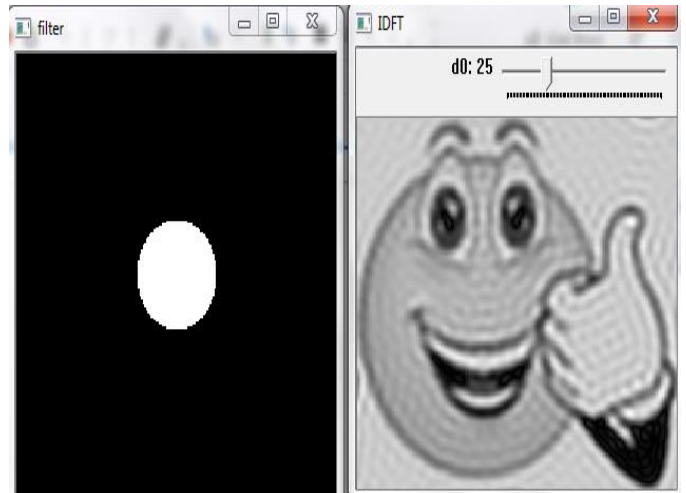
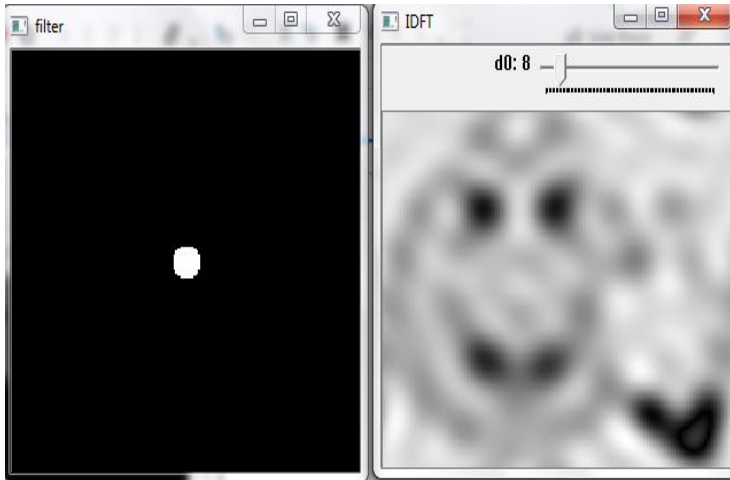


X



The small d_0 the more blurred you get

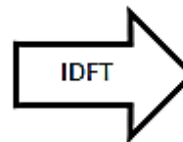
In order to remove spark noise and get less blurring you need to choose a suitable d_0 that is not so small or too large



3. Inverse Fourier

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(ux/M + vy/N)}$$

54	-13.5+16.4j	-13.5+16.4j
-4.5-9.5j	15+1.7j	-24+5.1j
-4.5+9.5j	-24-5.1j	15-1.7j



0	5	10
5	0	20
4	8	2

Implementation

1. Image setup

- Expand the image to an optimal size.
- Convert image into floating and normalizing it in range of 0 : 1
- Make place for both the complex and the real values.

2. DFT.

- Apply DFT
- Rearrange frequencies

3. Filter

- Filter function
- Apply filter

4. IDFT

- Apply IDFT
- Calculate magnitude and normalize image in range of 0 : 1

Image setup

Expand the image to an optimal size.

```
Mat I = imread("x2.jpg", 0);
int m = getOptimalDFTSize(I.rows);
int n = getOptimalDFTSize(I.cols);
Mat padded;
copyMakeBorder(I, padded, 0, m - I.rows, 0, n - I.cols, 0, Scalar(0));
```

Convert image into floating and normalizing it in range of 0 : 1

```
padded.convertTo(padded, CV_32FC1, 1.0 / 255.0);
```

Make place for both the complex and the real values.

```
Mat planes[2] = {padded, Mat::zeros(padded.size(), CV_32FC1) };
Mat complexI;
merge(planes, 2, complexI);
```

DFT.

Apply DFT

```
dft(complexI, complexI);
```

Rearrange frequencies

```
int cx = complexI.cols / 2;
int cy = complexI.rows / 2;
Mat p1(complexI, Rect(0, 0, cx, cy));
Mat p2(complexI, Rect(cx, 0, cx, cy));
Mat p3(complexI, Rect(0, cy, cx, cy));
Mat p4(complexI, Rect(cx, cy, cx, cy));
Mat temp;
p1.copyTo(temp);
p4.copyTo(p1);
temp.copyTo(p4);
p2.copyTo(temp);
p3.copyTo(p2);
temp.copyTo(p3);
```

Filter

Filter function

```
int d0 = 100;
for (int i = 0; i < Lfilter.rows; i++)
    for (int j = 0; j < Lfilter.cols; j++)
    {
        double z1 = i - Lfilter.rows / 2;
        double z2 = j - Lfilter.cols / 2;
        if (sqrt(pow(z1, 2) + pow(z2, 2)) <= d0)
            Lfilter.at<float>(i, j) = 1;
        else
            Lfilter.at<float>(i, j) = 0;
    }
```

Apply filter

```
split(complexI, planes);
Mat outR, outI;
multiply(planes[0], Lfilter, outR);
multiply(planes[1], Lfilter, outI);
Mat out_planes [2] = { outR, outI };
Mat out_complexI;
merge(out_planes, 2, out_complexI);
```

IDFT

Apply IDFT

```
idft(out_complexI, out_complexI);
```

Calculate magnitude and normalize image in range of 0 : 1

```
split(out_complexI, planes);
Mat out;
magnitude(planes[0], planes[1], out);
normalize(out, out, 1, 0, CV_MINMAX);
imshow("after IDFT", out);
```

```

int main()
{
    Mat src = imread("med2.jpg", 0);
    namedWindow("image", 0);
    imshow("image", src);
    int m = getOptimalDFTSize(src.rows);
    int n = getOptimalDFTSize(src.cols);
    Mat padded;
    copyMakeBorder(src, padded, 0, m - src.rows, 0, n - src.cols, 0);
    padded.convertTo(padded, CV_32FC1, 1.0 / 255.0);
    Mat planes[2] = { padded, Mat::zeros(padded.size(), CV_32FC1) };
    Mat complexI;
    merge(planes, 2, complexI);
    dft(complexI, complexI);
    //////////////////////////////////////
    int cx = complexI.cols / 2;
    int cy = complexI.rows / 2;
    Mat p1(complexI, Rect(0, 0, cx, cy));
    Mat p2(complexI, Rect(cx, 0, cx, cy));
    Mat p3(complexI, Rect(0, cy, cx, cy));
    Mat p4(complexI, Rect(cx, cy, cx, cy));
    Mat temp;
    p1.copyTo(temp);
    p4.copyTo(p1);
    temp.copyTo(p4);
    p2.copyTo(temp);
    p3.copyTo(p2);
    temp.copyTo(p3);
    //////////////////////////////////////
    Mat Lfilter(complexI.size(), CV_32FC1);
    namedWindow("after IDFT", 0);
    createTrackbar("d0", "after IDFT", &d0, 200);
    for (int i = 0; i < Lfilter.rows; i++)
        for (int j = 0; j < Lfilter.cols; j++)
        {
            double z1 = i - Lfilter.rows / 2;
            double z2 = j - Lfilter.cols / 2;
            if (sqrt(pow(z1, 2) + pow(z2, 2)) <= d0)
                Lfilter.at<float>(i, j) = 1;
            else
                Lfilter.at<float>(i, j) = 0;
        }
    namedWindow("filter", 0);
    imshow("filter", Lfilter);
    split(complexI, planes);
    Mat outR, outI;
    multiply(planes[0], Lfilter, outR);
    multiply(planes[1], Lfilter, outI);
    Mat out_planes[2] = { outR, outI };
    Mat out_complexI;
    merge(out_planes, 2, out_complexI);
    idft(out_complexI, out_complexI);
    split(out_complexI, planes);
    Mat out;
    magnitude(planes[0], planes[1], out);
    normalize(out, out, 1, 0, CV_MINMAX);
    imshow("after IDFT", out);
    waitKey(0);
}

```

